

Topic I: Sorting III – Heap Sort & Non-Comparison Sorts

Heap Sort Mechanics, Stability, Counting Sort, Radix Sort Conceptual Invariants

Part 1: Official & Canonical PYQs (Questions 1 to 3)

Q1. In-place Heap Sort Comparisons & Space Complexity

GATE CS/IT 2017 | MCQ

Which of the following statements is/are **CORRECT** regarding Heap Sort on an array of n distinct elements?

- (A) Worst-case time is $\Theta(n \log n)$, auxiliary space is $O(1)$.
 (B) Worst-case time is $\Theta(n^2)$, auxiliary space is $O(1)$.
 (C) Heap sort is a stable sorting algorithm.
 (D) Best-case running time is strictly $\Theta(n)$.

Answer: (A) Step-by-step Solution: Heap sort performs *Build-Max-Heap* in $O(n)$ time followed by $(n - 1)$ calls to *Max-Heapify*, each costing $O(\log n)$. Total worst, average, and best-case time complexity is $\Theta(n \log n)$ using iteratively implemented heapify, which requires $O(1)$ auxiliary memory. Heap sort is inherently *unstable* because swap operations during deletion destroy initial relative ordering of identical keys.

Q2. Counting Sort Preconditions and Asymptotics

GATE CS 2006 | MCQ

Let A be an array of n integers where every element satisfies $1 \leq A[j] \leq k$. Counting sort runs in linear time $\Theta(n)$ under which condition?

- (A) $k = O(n)$
 (B) $k = O(n \log n)$
 (C) $k = \Theta(n^2)$
 (D) k can be arbitrarily large.

Answer: (A) Step-by-step Solution: The standard counting sort algorithm executes in $\Theta(n + k)$ time, where $\Theta(k)$ is required to initialize and scan the frequency count buffer and $\Theta(n)$ is spent placing elements into the output array. For the time to be strictly linear $\Theta(n)$, k must satisfy $k = O(n)$.

Q3. Radix Sort Stability Requirement

GATE CS 2014 | MCQ

Why must the intermediate sub-sorting subroutine utilized in Least-Significant-Digit (LSD) Radix Sort be stable?

- (A) To guarantee an in-place memory allocation profile.
 (B) To preserve relative ordering resolved by lower-order digits.
 (C) To ensure worst-case time complexity does not exceed $O(\log n)$.
 (D) To enable parallel execution across multiple cores.

Answer: (B) Step-by-step Solution: LSD Radix Sort processes keys column-by-column from right to left. When two keys possess identical digits at significant position d , stability guarantees that their relative ordering—already correctly sorted by positions 0 to $d - 1$ —remains invariant. If an unstable sort were used, earlier sorting progress would be scrambled.

Q4. Lower Bound for Comparison Sorts vs Radix Sort

GATE CS 2009 | MCQ

The decision-tree model dictates that any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons. Radix sort sorts n integers in the range $[0, n^3 - 1]$ in $O(n)$ time because:

- (A) It eliminates comparison operations entirely using array indexing. (B) It performs divide-and-conquer using median-of-medians.
(C) Its decision tree possesses degree higher than binary. (D) It works exclusively on floating point numeric registers.

Answer: (A) Step-by-step Solution: The $\Omega(n \log n)$ lower bound applies strictly to models where decisions are limited to binary comparisons ($A[i] \leq A[j]$). Radix Sort uses positional digit extraction and direct memory addressing (typically via counting sort in base n), bypassing the comparison tree entirely. Representing numbers in base n yields $d = 3$ digits, giving runtime $\Theta(3(n + n)) = \Theta(n)$.

Q5. Stability Classification of Sorting Algorithms

GATE CS 2011 / DA 2024 Sample | MSQ

Which of the following sorting algorithms is/are inherently **NOT STABLE** in their standard textbook formulations? [Select all that apply]

- (A) Heap Sort (B) Counting Sort
(C) Merge Sort (D) Quick Sort

Answer: (A) and (D) Step-by-step Solution: (A) Heap Sort swaps root with the end leaf, violating order for duplicate keys. (D) Standard Quick Sort partitions elements across the pivot via non-adjacent swaps, disrupting original relative positions. Merge Sort and Counting Sort are both inherently stable when standard backward/stable scans are implemented.

Q6. Radix Sort Passes for Hexadecimal Representation

GATE CS 2015 | NAT

Consider sorting 1000 integers where each integer is represented as a 16-bit unsigned binary number. If Radix Sort is configured using radix/base $b = 16$ (hexadecimal digits), what is the exact number of counting sort passes required?

Answer: 4 Step-by-step Solution: Each 16-bit integer consists of 16 bits. With base $b = 16 = 2^4$, each digit represents 4 bits. The number of digits is $d = \lceil 16 / 4 \rceil = 4$. Hence, LSD radix sort performs exactly 4 stable counting sort passes over the array.

Q7. Radix Sort Optimal Base Formulation

GATE CS 2004 | MCQ

To sort n keys in the range 1 to n^k using Radix Sort in $\Theta(n)$ time, the digit base r must be selected as:

- (A) $\Theta(1)$ (B) $\Theta(\log n)$
(C) $\Theta(n)$ (D) $\Theta(n^k)$

Answer: (C) Step-by-step Solution: Total runtime is $\Theta(d(n + r))$. When numbers are bounded by n^k , choosing base $r = n$ makes the number of digits $d = \log_r(n^k) = k$. For a constant k , total time becomes $\Theta(k(n + n)) = \Theta(n)$.

Q8. Counting Sort Output Offset Pointer

GATE CS 2018 | NAT

Given input array $A = [3, 1, 4, 1, 5, 9, 2, 6, 5]$ sorted using counting sort with range $[0, 9]$. After calculating prefix sums $C[i] = \sum_{j=0}^i \text{count}[j]$, what is the value of $C[4]$?

Answer: 5 Step-by-step Solution: Count occurrences of each key: 0:0, 1:2, 2:1, 3:1, 4:1, 5:2, 6:1, 7:0, 8:0, 9:1. The prefix sum array entries: $C[0]=0$, $C[1]=2$, $C[2]=3$, $C[3]=4$, $C[4]=5$. Elements ≤ 4 are $\{1, 1, 2, 3, 4\}$, totaling exactly 5 elements.

Q9. Cache Locality: Heap Sort vs Quick Sort

GATE CS 2021 | MCQ

Despite having guaranteed $O(n \log n)$ worst-case time, Heap Sort is often slower in practice than Quick Sort because:

- (A) Heap Sort requires recursive stack allocation of depth $O(n)$. (B) Child node accesses jump across memory locations, yielding poor cache locality.
(C) Heap Sort performs more element swaps than Selection Sort. (D) Max-heap property cannot be maintained on array buffers.

Answer: (B) Step-by-step Solution: In array representation, children of index i reside at $2i$ and $2i + 1$. Sifting down traverses memory exponentially (indices double each hop), causing widespread CPU cache misses. Quick Sort accesses sequential adjacent blocks during partitioning, maximizing L1/L2 cache hit rates.

Q10. In-Place Auxiliary Memory Constraints

GATE CS 2020 | MSQ

Which of the following sorting algorithms can be executed strictly **in-place** using $O(1)$ auxiliary space? [Select all that apply]

- (A) Heap Sort (iterative sift-down) (B) Counting Sort
(C) Radix Sort with counting sort subroutines (D) Selection Sort

Answer: (A) and (D) Step-by-step Solution: Heap Sort and Selection Sort overwrite input array positions directly with $O(1)$ auxiliary scalar pointers. Standard Counting Sort and LSD Radix Sort require separate count buffers and an auxiliary output array of size n , making auxiliary memory $\Theta(n + k) \neq O(1)$.

Q11. Radix Sort on String Keys of Variable Length

New Practice | MCQ

To sort n variable-length English alphabetic strings where the total number of characters across all strings is L , the most optimal linear approach using Radix Sort achieves time complexity:

- (A) $O(n \cdot \text{max_len})$ (B) $O(L + 26)$
(C) $O(L \log n)$ (D) $O(n^2 + L)$

Answer: (B) Step-by-step Solution: Grouping strings by length and applying Most-Significant-Digit (MSD) or reverse LSD bucket-chaining processes each character in an alphabet of size $\Sigma = 26$ exactly once, achieving $\Theta(L + \Sigma) = O(L + 26)$ time without padding strings to max length.

Q12. Counting Sort with Negative Value Range

[New Practice | NAT](#)

An array contains 20 integers in the continuous range $[-8, +12]$. What is the minimum size of the frequency counter array required to sort this array using Counting Sort?

Answer: 21 Step-by-step Solution: Minimum value is -8 and maximum is 12 . The number of distinct possible integer values is $(\max - \min + 1) = 12 - (-8) + 1 = 21$. By applying an offset key translation $k' = k - (-8)$, a frequency buffer of size 21 indexed from 0 to 20 suffices.

Q13. Heap Sort Number of Sift-Down Steps

[New Practice | NAT](#)

In an array of 7 elements already sorted in strictly descending order, a standard Max-Heap is built using bottom-up *Build-Max-Heap*. How many element swaps are performed during this build step?

Answer: 0 Step-by-step Solution: An array sorted descending satisfies $A[i] > A[2i]$ and $A[i] > A[2i + 1]$ at all parent indices $i \in \{3, 2, 1\}$. Every subtree already satisfies the max-heap property. Consequently, *Max-Heapify* terminates immediately at every node, executing 0 swaps.

Q14. Radix Sort Bit Chunk Optimization

[New Practice | MSQ](#)

Which of the following statements is/are **TRUE** regarding non-comparison sorting? [Select all that apply]

- (A) Radix sort can sort 32-bit IEEE-754 floating point numbers if sign-bit flips are handled. (B) Counting sort is an online sorting algorithm.
(C) Counting sort requires prior knowledge of the range of keys. (D) Radix sort does not require keys to be discretizable into ordered components.

Answer: (A) and (C) Step-by-step Solution: (A) IEEE-754 numbers maintain lexical ordering in binary; flipping the sign bit allows standard Radix Sort to order positive and negative floats. (C) It requires explicit upper/lower bounds to allocate the frequency buffer. (B) is false as it needs all keys before output generation. (D) is false because lexicographical decomposition is strictly necessary.

Q15. Bottom-Up Build-Heap Asymptotic Derivation

[GATE CS 2003 / 2017 | MCQ](#)

The tight asymptotic bound for building a binary max-heap of n elements using *Build-Max-Heap* is:

- (A) $\Theta(n \log n)$ (B) $\Theta(n)$
(C) $\Theta(\log n)$ (D) $\Theta(n^2)$

Answer: (B) Step-by-step Solution: A heap of n nodes has at most $\lceil n / 2^{h+1} \rceil$ nodes at height h . Total cost is bounded by $\sum_{h=0}^{\lceil \log n \rceil} \lceil n / 2^{h+1} \rceil$. $O(h) = O(n \sum_{h=0}^{\infty} h / 2^h)$. The infinite series converges to 2, yielding total cost $O(2n) = \Theta(n)$.

Topic II: Heap Patterns – Build, Heapify, Extract (PYQs 16-18 & Practice 19-20)

Q16. Max-Heap Deletion Array Contents

GATE CS 2009 | MCQ

Consider the max-heap array representation: [25, 14, 16, 13, 10, 8, 12]. What is the state of the array after performing **ONE** delete-max (extract-max) operation?

- (A) [16, 14, 12, 13, 10, 8] (B) [16, 14, 8, 13, 10, 12]
(C) [14, 13, 12, 10, 8, 16] (D) [16, 13, 14, 10, 8, 12]

Answer: (A) Step-by-step Solution: Extract root (25). Move last leaf (12) to root: [12, 14, 16, 13, 10, 8]. Run *Max-Heapify* at root (index 1): Left child is 14 (idx 2), right child is 16 (idx 3). Largest is 16; swap 12 and 16 → [16, 14, 12, 13, 10, 8]. Index 3 has left child 8 (idx 6) and $12 \geq 8$; heap property satisfied.

Q17. Swaps Performed During Build-Heap

GATE CS 2011 | NAT

Given the array $A = [4, 10, 3, 5, 1]$. What is the total number of element swaps executed when building a max-heap using the standard *Build-Max-Heap* algorithm?

Answer: 2 Step-by-step Solution: Array length $n = 5$. Non-leaf nodes start at $\lfloor 5 / 2 \rfloor = 2$ down to 1. (1) *Heapify* at index 2 (value 10): Children are 5, 1. Max is 10; 0 swaps. (2) *Heapify* at index 1 (value 4): Left child is 10, right is 3. Max is 10; swap 4 with 10 (Swap 1). Array becomes [10, 4, 3, 5, 1]. Sift-down continues at index 2 (value 4): Children are 5, 1. Max is 5; swap 4 with 5 (Swap 2). Total swaps = 2.

Q18. Insertion Sequence into Binary Max-Heap

GATE CS 2017 | MCQ

Starting with an empty binary max-heap, the sequence 10, 20, 15, 30, 40 is inserted one by one. The resulting heap stored as an array is:

- (A) [40, 30, 15, 10, 20] (B) [40, 30, 15, 20, 10]
(C) [40, 20, 30, 10, 15] (D) [40, 30, 20, 10, 15]

Answer: (A) Step-by-step Solution: Insert 10: [10]. Insert 20: sifts up → [20, 10]. Insert 15: [20, 10, 15]. Insert 30: placed at idx 4, parent idx 2 (10) < 30 → swap with 10 → parent idx 1 (20) < 30 → swap with 20 → [30, 20, 15, 10]. Insert 40: placed at idx 5, parent idx 2 (20) < 40 → swap with 20 → parent idx 1 (30) < 40 → swap with 30 → [40, 30, 15, 10, 20].

Q19. Finding k -th Largest using Min-Heap vs Max-Heap

New Practice | MCQ

To identify the k -th largest element from a streaming data flow of N elements (where $N \gg k$), the most space- and time-efficient approach maintains:

- (A) A Max-Heap of size N , taking $O(N + k \log N)$ time. (B) A Min-Heap of size k , taking $O(N \log k)$ time and $O(k)$ space.
(C) A Min-Heap of size N , taking $O(N \log N)$ time. (D) A Max-Heap of size k , taking $O(N \log k)$ time.

Answer: (B) Step-by-step Solution: A min-heap of size k stores the top k largest elements encountered so far. Root contains the smallest among the top k . For each incoming element, if it exceeds the root, replace root and heapify in $O(\log k)$. Space is strictly $O(k)$, ideal for large streams.

Q20. CSP Formal Definition & State Space Structure

GATE DA 2024 / CS AI | MCQ

A Constraint Satisfaction Problem (CSP) is formally defined as a tuple $\langle X, D, C \rangle$. In contrast to general state-space search (e.g., A^*), the standard goal test in a CSP checks:

- | | |
|--|--|
| (A) Whether a target path cost function has been minimized. | (B) Whether an assignment is complete and satisfies all constraints. |
| (C) Whether the depth limit in iterative deepening has been reached. | (D) Whether heuristic value $h(n)$ equals 0 at the frontier node. |

Answer: (B) Step-by-step Solution: A CSP state is represented as variable-value assignments. A state is a valid solution if and only if it is *complete* (every variable in X is assigned a value from its domain D) and *consistent* (no constraint in C is violated). The sequence of actions to reach the state is irrelevant.

Q21. Classification of Constraint Arity

GATE CS AI / UGC-NET 2020 | MSQ

Which of the following pairings between constraint statements and their arity classifications is/are **CORRECT**? [Select all that apply]

- | | |
|---|---|
| (A) $X_1 \neq \text{"Red"}$: Unary Constraint | (B) $X_1 \neq X_2$: Binary Constraint |
| (C) $AllDifferent(X_1, X_2, X_3, X_4)$: Higher-Order / Global Constraint | (D) $X_1 + X_2 = X_3$: Binary Constraint |

Answer: (A), (B), and (C) Step-by-step Solution: Arity equals the number of variables in the constraint scope. (A) involves 1 variable (unary). (B) involves 2 variables (binary). (C) involves 4 variables (global/higher-order). (D) involves 3 variables (X_1, X_2, X_3), making it a ternary (higher-order) constraint, not binary.

Q22. Hard vs Soft Constraints in Optimization

GATE DA Sample / AI Standard | MCQ

In a university timetable formulation, "No professor can teach two classes simultaneously" and "Faculty prefer morning slots over evening slots" represent:

- | | |
|---|---|
| (A) Both are Soft Constraints. | (B) Hard Constraint and Soft Constraint respectively. |
| (C) Soft Constraint and Hard Constraint respectively. | (D) Both are Unary Constraints. |

Answer: (B) Step-by-step Solution: A *hard constraint* must be unconditionally satisfied for an assignment to be valid (physical impossibility of being in two classrooms). A *soft constraint* expresses preferences and penalizes objective weights in Constrained Optimization Problems (COP).

Q23. Dual Graph Transformation of Higher-Order Constraints

New Practice | MCQ

Any non-binary (higher-order) CSP with discrete domains can be converted into an equivalent binary CSP by:

- | | |
|--|---|
| (A) Relaxing hard constraints into unconstrained penalties. | (B) Introducing auxiliary dual variables whose domains are tuples of the original relation. |
| (C) Applying Dijkstra's shortest path algorithm over the constraint network. | (D) Using the Minimum Remaining Values (MRV) heuristic without extra variables. |

Answer: (B) Step-by-step Solution: The dual graph transformation introduces a new variable for each non-binary constraint whose domain comprises all allowable value tuples of the original scope. Binary constraints then enforce that dual variables agree on shared variable assignments.

Q24. Map-Coloring Constraint Graph Representation

GATE CS 2008 / DA 2024 Benchmark | NAT

Consider a map of 5 regions A, B, C, D, E where borders exist between: $(A, B), (A, C), (B, C), (C, D), (D, E)$, and (B, E) . How many edges (binary constraints) exist in the constraint graph?

Answer: 6 Step-by-step Solution: In a CSP constraint graph for map coloring, each territorial variable is a node, and every adjacent border represents a binary inequality constraint ($X_u \neq X_v$). Listing the 6 mutual boundaries gives exactly 6 undirected edges.

Q25. N-Queens Problem Formulation as CSP

GATE CS AI / Standard Benchmark | MSQ

When formulating the N -Queens problem with variables $Q_1, Q_2, \dots, Q_N$ representing the row index of the queen in column i with domains $\{1, \dots, N\}$, which constraints are strictly required? [Select all that apply]

- (A) $Q_i \neq Q_j$ for all $i \neq j$ (No two queens in the same row) (B) $|Q_i - Q_j| \neq |i - j|$ for all $i \neq j$ (No two queens on the same diagonal)
- (C) $Q_i < Q_{i+1}$ for all $1 \leq i < N$ (D) $\sum_{i=1}^N Q_i = N(N+1)/2$

Answer: (A) and (B) Step-by-step Solution: Placing queen i in column i inherently prevents column conflicts. To prevent row conflicts, $Q_i \neq Q_j$. To prevent diagonal attacks, the vertical distance cannot equal the horizontal distance: $|Q_i - Q_j| \neq |i - j|$. (C) and (D) are invalid artificial restrictions.

Q26. Cryptarithmic Column Addition Constraints

GATE AI Canonical | MCQ

In the cryptarithmic puzzle $TWO + TWO = FOUR$, let carry bits be $C_1, C_2, C_3 \in \{0, 1\}$. The domain of the leading letter F is:

- (A) $\{0, 1, 2, \dots, 9\}$ (B) $\{1\}$
- (C) $\{1, 2\}$ (D) $\{0\}$

Answer: (B) Step-by-step Solution: The sum of two 3-digit numbers can at most be $999 + 999 = 1998$. The carry-out C_3 from the hundred's column directly produces the leading digit F . Because a leading letter cannot be zero ($F \neq 0$), the carry bit must be 1. Hence, $F = C_3 = 1$.

Q27. Map Coloring Number of Valid Assignments

New Practice | NAT

A triangle graph K_3 of regions X, Y, Z is mutually adjacent. Using a color palette $D = \{\text{Red, Green, Blue, Yellow}\}$ of size 4, how many distinct conflict-free valid assignments satisfy this CSP?

Answer: 24 Step-by-step Solution: Region X can be assigned any of the 4 colors. Neighbor Y cannot share X 's color, giving $(4 - 1) = 3$ options. Neighbor Z is adjacent to both X and Y , so it must differ from both distinct colors, leaving $(4 - 2) = 2$ choices. Total valid assignments = $4 \times 3 \times 2 = 24$.

Topic IV: Problem Formulation as CSPs (Practice Questions 28 to 30)

Q28. Sudoku Formulation as a Binary CSP

[New Practice | NAT](#)

In a standard 9×9 Sudoku grid formulated as a CSP with 81 variables (one per cell), how many binary inequality constraints ($X_i \neq X_j$) exist across all rows, columns, and 3×3 sub-grids?

Answer: 810 Step-by-step Solution: Each cell must differ from cells in its row, column, and block. Each cell shares an all-different unit with: 8 other row cells + 8 other column cells + 4 other block cells (excluding the 4 already counted in row/column intersections) = 20 peers. In an undirected graph where each node has degree 20, the number of unique edges is $(81 \times 20) / 2 = 810$ binary constraints.

Q29. Heuristic Selection in CSP Backtracking

[New Practice | MSQ](#)

Which of the following statements is/are **TRUE** regarding heuristics utilized in CSP search? [Select all that apply]

- | | |
|--|--|
| (A) The Minimum Remaining Values (MRV) heuristic chooses the variable with fewest legal values. | (B) The Degree heuristic breaks ties by choosing the variable with most constraints on remaining unassigned variables. |
| (C) The Least Constraining Value (LCV) heuristic prefers the value that rules out the maximum choices for neighboring variables. | (D) Arc Consistency (AC-3) detects failure earlier than forward checking. |

Answer: (A), (B), and (D) Step-by-step Solution: (A) and (B) are the textbook "fail-first" variable selection rules. (D) is true because AC-3 propagates constraints transitively across the entire network, whereas forward checking only checks immediate neighbors. (C) is false: LCV chooses the value that rules out the *fewest* values for neighbors (fail-last).

Q30. Graph Planarity and Four-Coloring Bound

[New Practice | MCQ](#)

A geographical region map formulated as a CSP is planar. If every variable domain contains 4 colors, the Four Color Theorem guarantees that:

- | | |
|--|---|
| (A) AC-3 alone will find the solution without any backtracking search. | (B) At least one conflict-free complete assignment always exists. |
| (C) The constraint graph cannot contain cycles of odd length. | (D) Backtracking search will run in polynomial time $O(N^4)$. |

Answer: (B) Step-by-step Solution: The Four Color Theorem guarantees that every loopless planar graph is 4-colorable. Therefore, for any planar map-coloring CSP with 4 available colors per domain, the solution space is guaranteed non-empty (at least one valid assignment exists). However, computing it remains NP-complete in the general graph case, and AC-3 does not guarantee zero backtracking.